



Congratulations!

You've completed Lesson 7: Coordinate multiple agents!

You can now build multi-agent systems where specialized agents work together to solve complex problems.

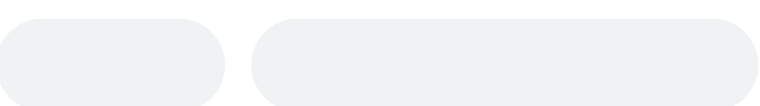
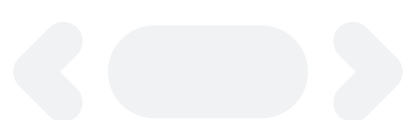
What you learned

Part 1: Understanding multi-agent systems

- What multi-agent systems are
- When to use multi-agent vs. single-agent
- Parent-child hierarchy with `sub_agents`

Python

```
# Multi-agent foundation
root_agent = LlmAgent(
    name='coordinator',
    sub_agents=[billing_specialist, technical_specialist]
)
```



Part 2: Workflow agents

- **SequentialAgent** - Run agents in order ($A \rightarrow B \rightarrow C$)
- **ParallelAgent** - Run agents at the same time ($A + B + C$)
- **LoopAgent** - Repeat agents ($A \rightarrow B$, repeat)

Python

```
# Workflow orchestration
root_agent = SequentialAgent(
    sub_agents=[intake, parallel_research, response]
)
```

Part 3: Agent communication

- Save output with **output_key**
- Read output with **{key}** templating
- Data flows through pipelines via state

Python

```
# State communication
researcher = LlmAgent(output_key='research_data', ...)
writer = LlmAgent(instruction='Use: {research_data}', ...)
```

Part 4: Building your first multi-agent app

- Combined all concepts in a capstone project
- Built a complete customer service multi-agent app
- Working reference code for future projects

Quick reference

When to use each workflow

Workflow	Use When
Sequential	Steps depend on each other
Parallel	Steps are independent (faster)
Loop	Need to repeat/refine

State communication pattern

```
Python
# Agent A writes
agent_a = LlmAgent(output_key='data', ...)

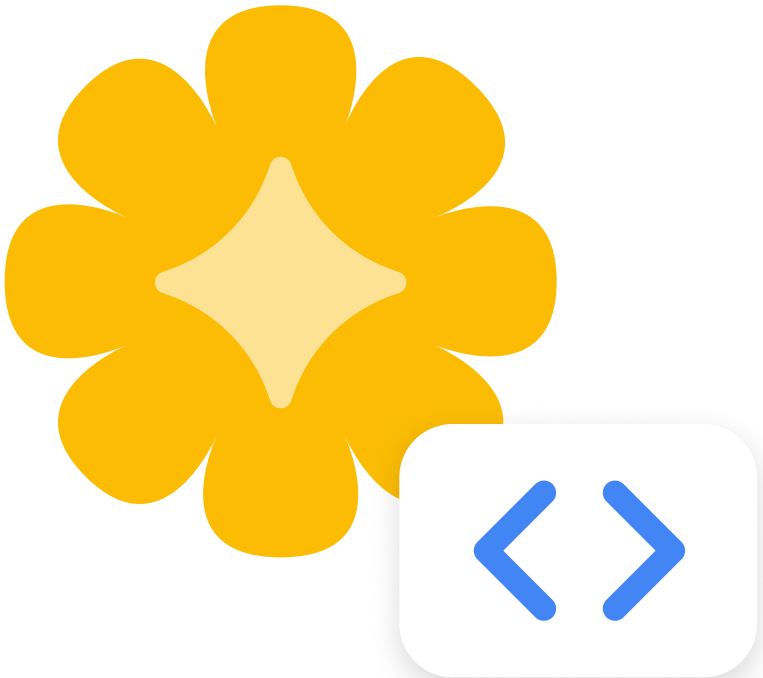
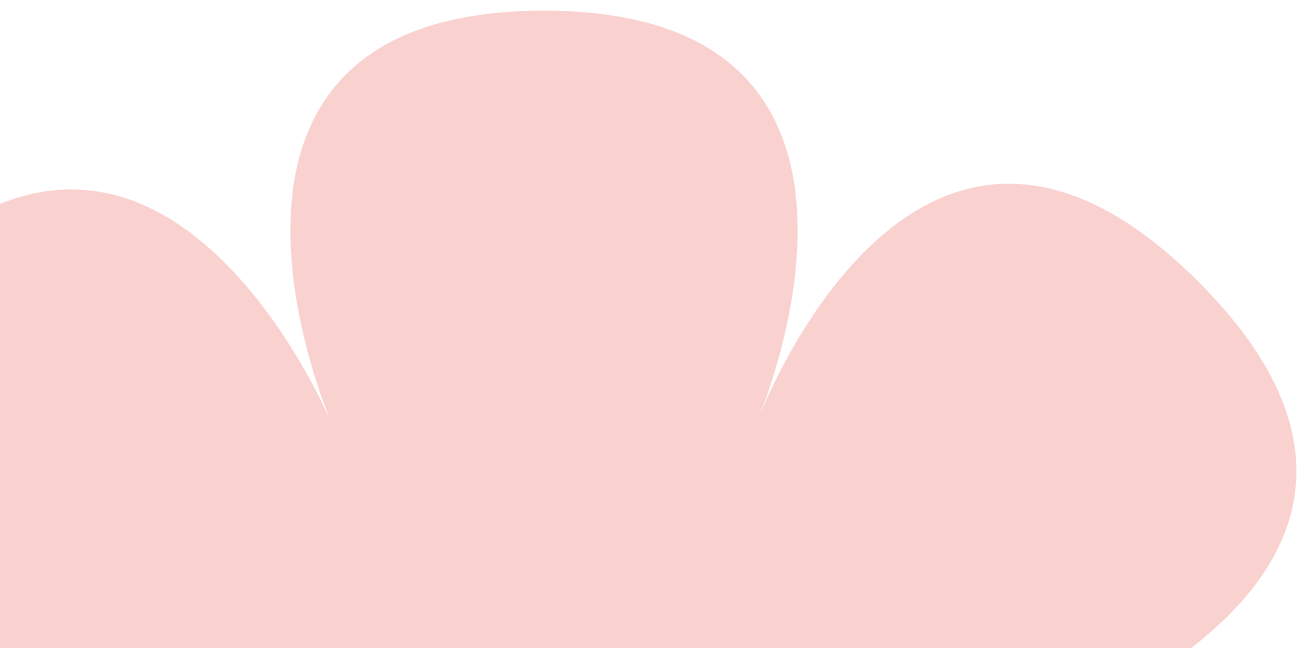
# Agent B reads
agent_b = LlmAgent(instruction='Use {data}', ...)
```

Key points to remember

- Use distinct **output_key** for parallel agents (avoid conflicts)
- Always set **max_iterations** on LoopAgent
- Each agent should have one focused role
- Start simple, add complexity only when needed

What’s next?

- Lesson 8: Orchestrate Complex Workflows - Advanced patterns with state
- Lesson 9: Deploy Your First Agent - Take agents to production



Resources

Primary references

- 📖 [Multi-Agent Systems](#) - Core multi-agent architecture and patterns
- 🔧 [Workflow Agents Overview](#) - Sequential, parallel, and loop agents
- 📖 [SequentialAgent](#) - Ordered execution patterns
- 📖 [ParallelAgent](#) - Concurrent execution patterns
- 📖 [LoopAgent](#) - Iterative execution patterns

From the community

- 📖 [Fun with agentic workflows in ADK](#) - Exploration of sequential, parallel, and loop workflows with practical code examples (part 2)
- 🤖 [demo-multi-agent-google-adk](#) - Working repository demonstrating multi-agent architecture and agent coordination (parts 1, 3, 4)
- 📖 [Building Intelligent Multi-Agent Systems with Google's ADK](#) - How to create specialized agents that collaboratively solve complex problems (parts 1, 4)
- 📺 [Building Curated Content with AI Agents: ADK Workflow](#) - Complete multi-agent workflow with Orchestrator, Researcher, Writer, Critic, and Reviser agents (part 2)
- 📖 [Connecting Salesforce Agentforce with ADK](#) - A2A protocol integration for agent-to-agent communication across systems (part 3)

❓ Questions or feedback?
Share in the [ADK Community Forum](#)

Well done on completing lesson 7!